Karishma Premnath
EECE 7205, Stacks and Queues, In-Class Work

Problem 1. Expression Evaluation Using Stacks

(a) Functions

- `isNumber(s)` a token is a number if it is all digits, or a minus sign followed by digits. This is how `-7` is read as a value and not as subtraction.
- `toInt(s)` converts a number token to an `int`, handling the leading minus sign.
- `precedence(c)` `^` is 3, `*` and `/` are 2, `+` and `-` are 1.
- `floorDiv(a, b)` rounds down toward negative infinity by subtracting one when the signs differ and the division is not exact.
- `applyOp(a, b, c)` does one operation on two integers, with `a` as the left operand. Division calls `floorDiv`.
- `evaluate(tok)` pushes values on the values stack, and before pushing an operator it applies every waiting operator that must go first.
- `ValueStack` and `OpStack` the two stacks, one holding `int` values and one holding `char` operators.

(b) Evaluate `8 + 3 * 2 ^ 2 - 20 / 4`

Step	Operator	Calculation	Expression left
			`8 + 3 * 2 ^ 2 - 20 / 4`
1	`^`	`2 ^ 2 = 4`	`8 + 3 * 4 - 20 / 4`
2	`*`	`3 * 4 = 12`	`8 + 12 - 20 / 4`
3	`/`	`20 / 4 = 5`	`8 + 12 - 5`
4	`+`	`8 + 12 = 20`	`20 - 5`
5	`-`	`20 - 5 = 15`	`15`

Step 1 goes first because `^` has the highest precedence. Steps 2 and 3 come next and are taken left to right, then steps 4 and 5 the same way.

Answer: 15

(c) Trace

The two stacks after each token is processed:

Token	Action	Values (bottom to top)	Operators
`8`	push 8	8	
`+`	push `+`	8	`+`
`3`	push 3	8 3	`+`
`*`	push `*`	8 3	`+ *`
`2`	push 2	8 3 2	`+ *`
`^`	push `^`	8 3 2	`+ * ^`
`2`	push 2	8 3 2 2	`+ * ^`
`-`	apply `^`, `*`, `+`, then push `-`	20	`-`
`20`	push 20	20 20	`-`
`/`	push `/`	20 20	`- /`
`4`	push 4	20 20 4	`- /`

At token `-`, three operators are applied because `-` has lower precedence than all three operators on the stack:

Apply	Calculation	Values after
`^`	`2 ^ 2 = 4`	8 3 4
`*`	`3 * 4 = 12`	8 12
`+`	`8 + 12 = 20`	20

Remaining calculations once the tokens are finished:

Apply	Calculation	Values after
`/`	`20 / 4 = 5`	20 5
`-`	`20 - 5 = 15`	15

Result: 15

(d) Tests

Expression	Answer	Common wrong answer
`2 ^ 3 ^ 2`	512	64
`-7 / 2`	-4	-3

`2 ^ 3 ^ 2` gives **512**. When the second `^` arrives, the first one has equal precedence, and `^` is right associative, so it is left waiting instead of being applied. That makes `3 ^ 2 = 9` run first, then `2 ^ 9 = 512`. Treating `^` as left associative would give `(2 ^ 3) ^ 2 = 64`.

`-7 / 2` gives **-4**. The exact value is -3.5. C++ integer division cuts the fraction off toward zero and gives -3, but the question asks for rounding down toward negative infinity, which is -4. `floorDiv` subtracts one because the signs differ and the division is not exact. A clean division such as $-20/4$ is untouched and stays -5.

Code

```
// Q1: infix expression evaluation using two stacks

#include <iostream>
#include <string>
#include <vector>
using namespace std;

const int MAX = 100;

class ValueStack
{
    int a[MAX];
    int t = -1;
public:
    bool empty() { return t == -1; }
    bool full() { return t == MAX - 1; }
    void push(int x) { if (!full()) a[++t] = x; }
    int pop() { return a[t--]; }
};

class OpStack
{
    char a[MAX];
    int t = -1;
public:
    bool empty() { return t == -1; }
    bool full() { return t == MAX - 1; }
    char top() { return a[t]; }
    void push(char c) { if (!full()) a[++t] = c; }
    char pop() { return a[t--]; }
};

bool isNumber(string s)
{
    int i = 0;
    if (s[0] == '-' && s.size() > 1) i = 1;
    while (i < (int)s.size())
    {
        if (s[i] < '0' || s[i] > '9') return false;
        i++;
    }
    return true;
}

int toInt(string s)
{
    int n = 0, i = 0;
    bool neg = false;
    if (s[0] == '-') { neg = true; i = 1; }
    while (i < (int)s.size())
    {
        n = n * 10 + (s[i] - '0');
        i++;
    }
    if (neg) n = -n;
    return n;
}
```

```
int precedence(char c)
{
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    return 1;
}

int floorDiv(int a, int b)
{
    int q = a / b;
    if (a % b != 0 && (a < 0) != (b < 0)) q--;
    return q;
}

int applyOp(int a, int b, char c)
{
    if (c == '+') return a + b;
    if (c == '-') return a - b;
    if (c == '*') return a * b;
    if (c == '/') return floorDiv(a, b);
    int p = 1;
    for (int i = 0; i < b; i++) p = p * a;
    return p;
}

int evaluate(vector<string> tok)
{
    ValueStack vals;
    OpStack ops;

    for (int i = 0; i < (int)tok.size(); i++)
    {
        if (isNumber(tok[i]))
        {
            vals.push(toInt(tok[i]));
        }
        else
        {
            char c = tok[i][0];
            int pc = precedence(c);
            while (!ops.empty() && (precedence(ops.top()) > pc ||
                                    (precedence(ops.top()) == pc && c != '^')))
            {
                char o = ops.pop();
                int b = vals.pop();
                int a = vals.pop();
                vals.push(applyOp(a, b, o));
            }
            ops.push(c);
        }
    }

    while (!ops.empty())
    {
        char o = ops.pop();
        int b = vals.pop();
        int a = vals.pop();
        vals.push(applyOp(a, b, o));
```

```
    }

    return vals.pop();
}

int main()
{
    vector<string> e1 = {"8", "+", "3", "*", "2", "^", "2", "-", "20", "/", "4"};
    vector<string> e2 = {"2", "^", "3", "^", "2"};
    vector<string> e3 = {"-7", "/", "2"};

    cout << "(b) 8 + 3 * 2 ^ 2 - 20 / 4 = " << evaluate(e1) << endl;
    cout << "(d) 2 ^ 3 ^ 2              = " << evaluate(e2) << endl;
    cout << "(d) -7 / 2                 = " << evaluate(e3) << endl;
    return 0;
}
```

Output

```
(b) 8 + 3 * 2 ^ 2 - 20 / 4 = 15
(d) 2 ^ 3 ^ 2              = 512
(d) -7 / 2                 = -4
```

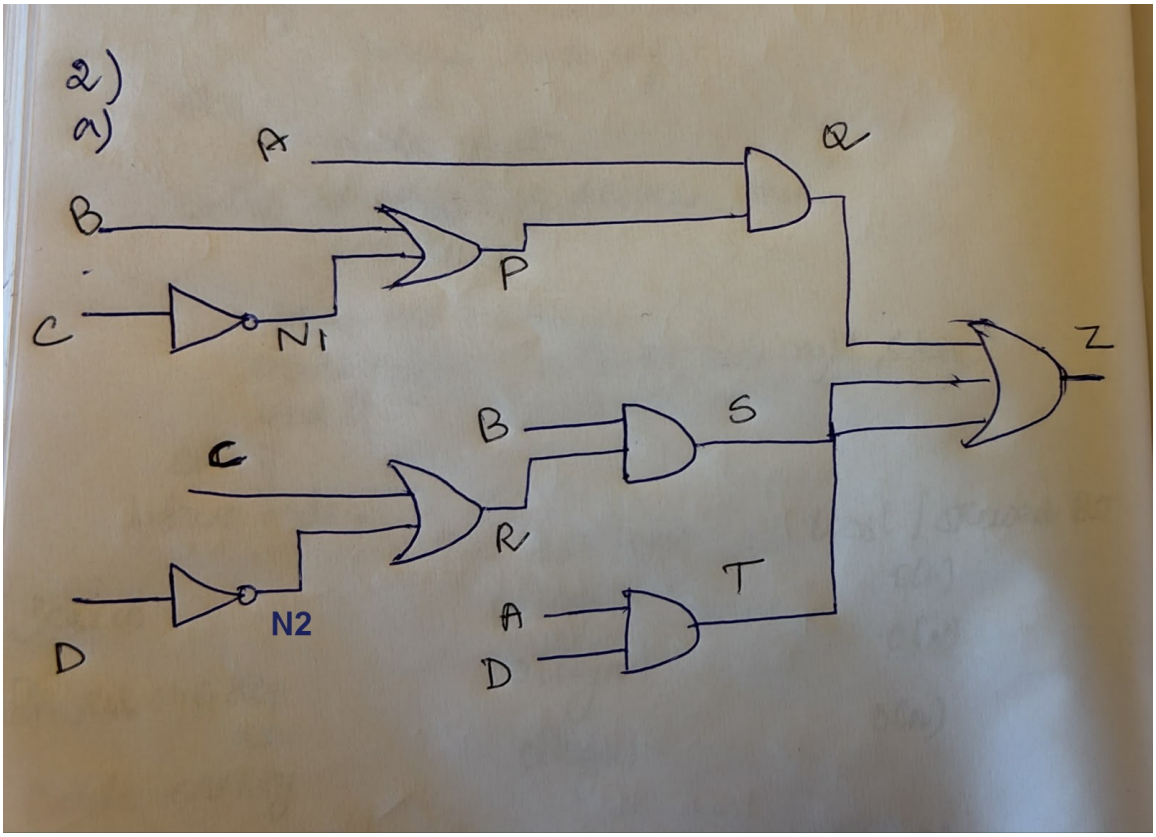
Problem 2. Logic Gate Analysis and Implementation

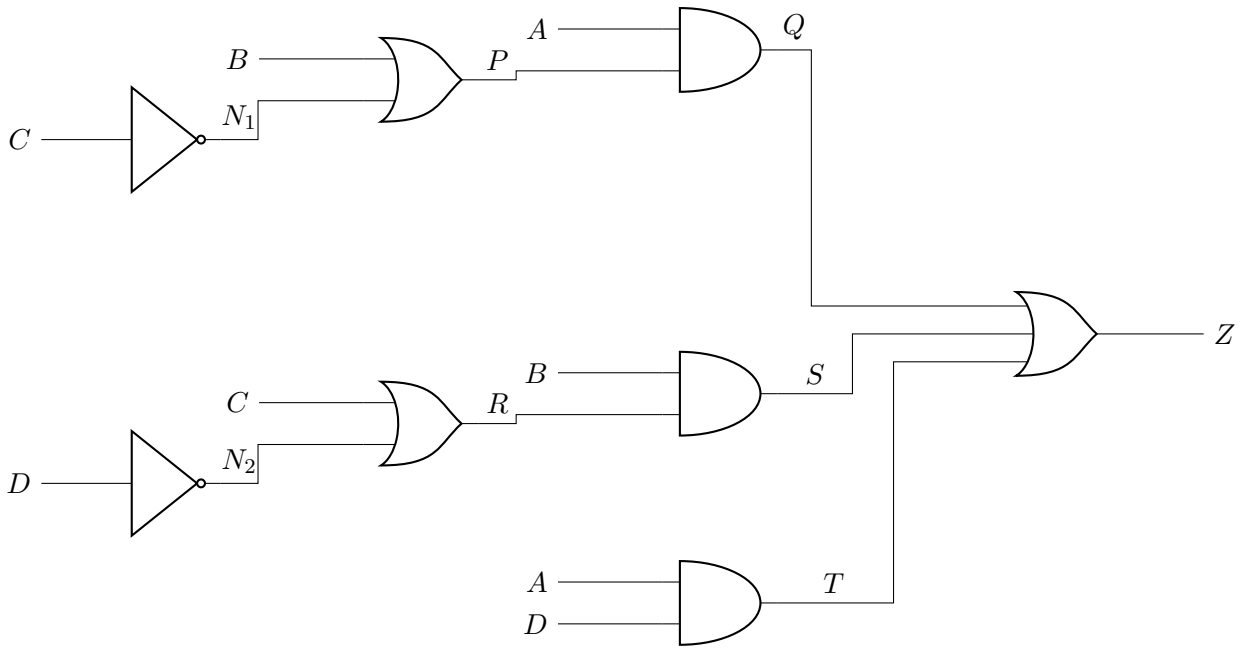
(a) Circuit and intermediate signals

Four levels, eight gates:

Level	Gate	Inputs	Output label
1	NOT	C	N_1
1	NOT	D	N_2
2	OR	B, N_1	P
2	OR	C, N_2	R
3	AND	A, P	Q
3	AND	B, R	S
3	AND	A, D	T
4	OR (3 input)	Q, S, T	Z

Each input is used twice: A feeds Q and T, B feeds P and S, C feeds N_1 and R, D feeds N_2 and T.





(b) Boolean expression

Working through the gate list in order:

$$
\begin{aligned}
N_1 &= C' \\
N_2 &= D' \\
P &= B + C' \\
Q &= A(B + C') \\
R &= C + D' \\
S &= B(C + D') \\
T &= AD \\
Z &= Q + S + T = A(B + C') + B(C + D') + AD
\end{aligned}
$$

Multiplied out:

$$Z = AB + AC' + BC + BD' + AD$$

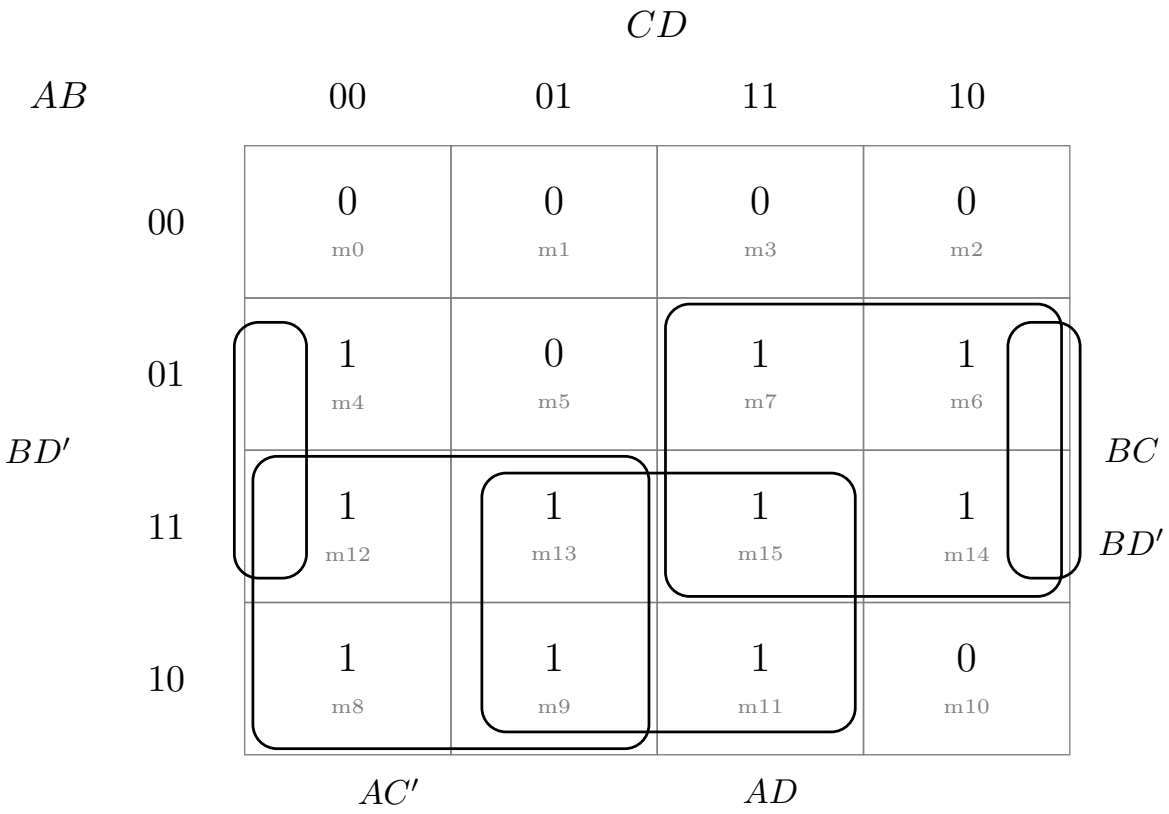
(c) Truth table

A	B	C	D	N_1	N_2	P	Q	R	S	T	Z
0	0	0	0	1	1	1	0	1	0	0	**0**
0	0	0	1	1	0	1	0	0	0	0	**0**
0	0	1	0	0	1	0	0	1	0	0	**0**
0	0	1	1	0	0	0	0	1	0	0	**0**
0	1	0	0	1	1	1	0	1	1	0	**1**
0	1	0	1	1	0	1	0	0	0	0	**0**
0	1	1	0	0	1	1	0	1	1	0	**1**
0	1	1	1	0	0	1	0	1	1	0	**1**
1	0	0	0	1	1	1	1	1	0	0	**1**
1	0	0	1	1	0	1	1	0	0	1	**1**
1	0	1	0	0	1	0	0	1	0	0	**0**
1	0	1	1	0	0	0	0	1	0	1	**1**
1	1	0	0	1	1	1	1	1	1	0	**1**
1	1	0	1	1	0	1	1	0	0	1	**1**
1	1	1	0	0	1	1	1	1	1	0	**1**
1	1	1	1	0	0	1	1	1	1	1	**1**

$Z = 1$ for ten input combinations: m_4, m_6, m_7, m_8, m_9, m_{11}, m_{12}, m_{13}, m_{14}, m_{15}.

(d) Karnaugh map and minimum SOP

Rows AB and columns CD, both in Gray-code order, with the four groups looped:



Four groups of four cells:

Group	Cells	Term
$B = 1$, $D = 0$	m_4, m_6, m_{12}, m_{14}	BD'
$B = 1$, $C = 1$	m_6, m_7, m_{14}, m_{15}	BC
$A = 1$, $D = 1$	m_9, m_{11}, m_{13}, m_{15}	AD
$A = 1$, $C = 0$	m_8, m_9, m_{12}, m_{13}	AC'

$$\boxed{Z = BD' + BC + AD + AC'}$$

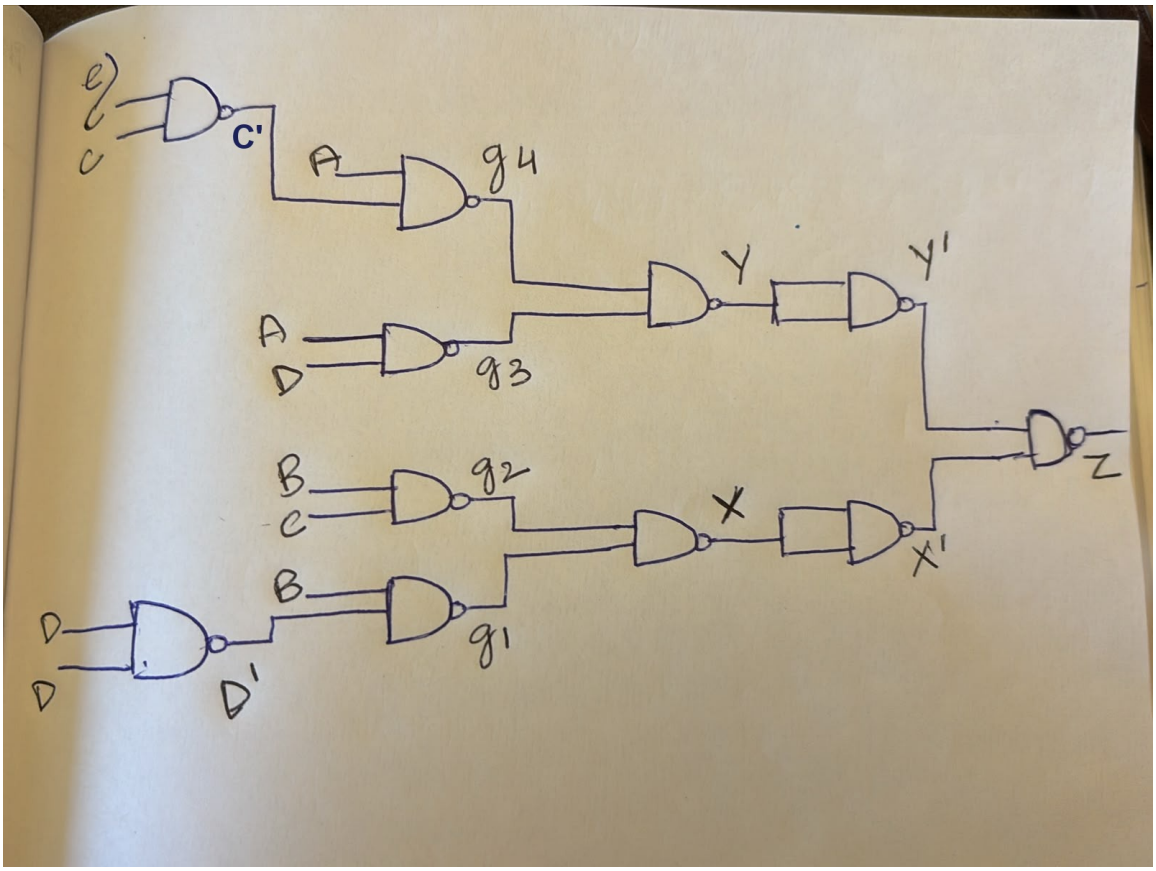
The BD' group wraps around the map. Its four cells are the two in the $CD = 00$ column and the two in the $CD = 10$ column. Those columns are next to each other in Gray-code order, so the group is legal even though it is drawn at the two edges.

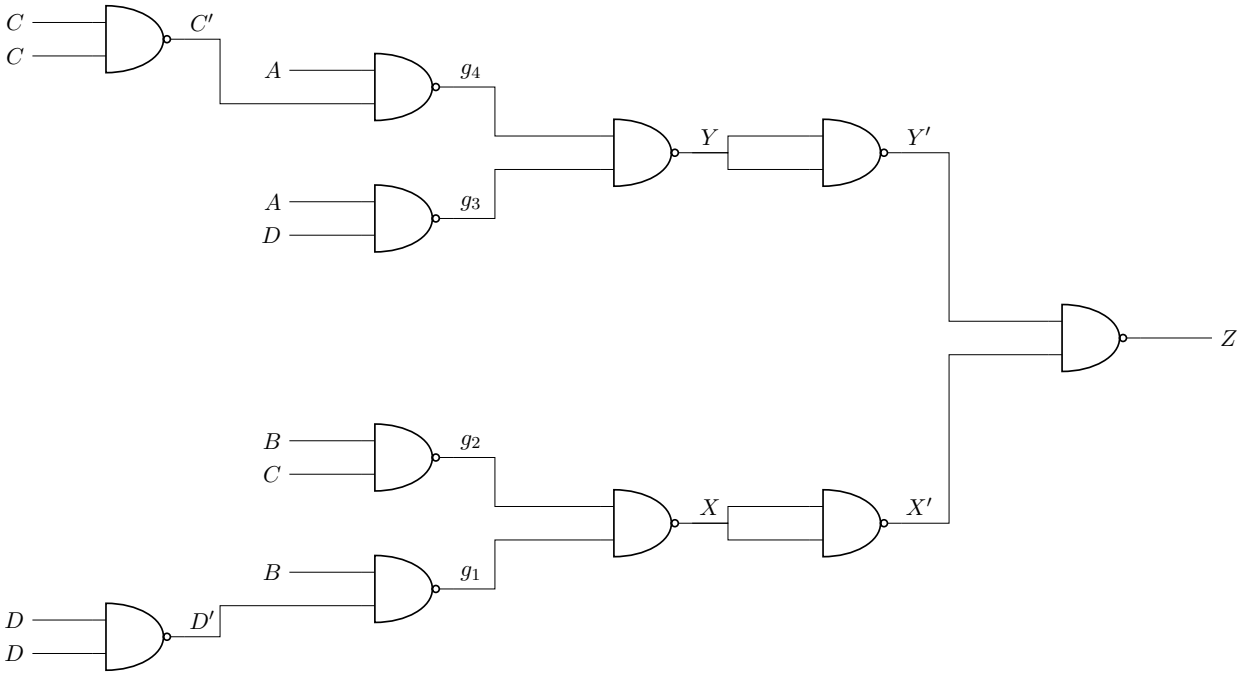
AB is also a prime implicant, covering m_{12}, m_{13}, m_{14}, m_{15}, but it is redundant because all four of those cells are already covered by the groups above. This four-term answer is the only minimum SOP.

(e) NAND-only implementation

Starting from $Z = BD' + BC + AD + AC'$. Each product term becomes a NAND gate. The two partial sums are inverted, and a final NAND joins them. A NAND with both inputs tied together acts as an inverter. Eleven two-input NAND gates:

Gate	Inputs	Output	Meaning
1	C, C	C'	inverter
2	D, D	D'	inverter
3	B, D'	g_1	$(BD')'$
4	B, C	g_2	$(BC)'$
5	A, D	g_3	$(AD)'$
6	A, C'	g_4	$(AC')'$
7	g_1, g_2	X	$BD' + BC$
8	g_3, g_4	Y	$AD + AC'$
9	X, X	X'	inverter
10	Y, Y	Y'	inverter
11	X', Y'	Z	$X + Y$





The program on the last page simulates these eleven NAND gates alongside the gate network, in the `Z nand` column. The two agree on all sixteen rows.

(f) Worst-case propagation delay

Every NAND gate has a delay of 20 ns, so the delay of a path is 20 ns times the number of gates on it.

Path	Gates	Delay
$C \to C' \to g_4 \to Y \to Y' \to Z$	5	100 ns
$D \to D' \to g_1 \to X \to X' \to Z$	5	100 ns
$A \to g_3 \to Y \to Y' \to Z$	4	80 ns
$B \to g_2 \to X \to X' \to Z$	4	80 ns

The longest input-to-output path is five gates, so the worst-case propagation delay is

$$5 \times 20 \text{ ns} = 100 \text{ ns}$$

reached from inputs C and D, because those two have to pass through an inverter before reaching a product gate. Inputs A and B are used uncomplemented and reach the output in four gates.

Evaluating the expression with a stack

The minimized expression is a sum of products, so it needs no parentheses and runs on the same two-stack evaluator as Q1. The values stack holds 0 and 1, the operators stack holds `&` and `|`, and `&` has higher precedence than `|`. A token like `C'` is read as the complement of C.

```
part (b):  A & B | A & C' | B & C | B & D' | A & D
part (d):  B & D' | B & C | A & D | A & C'
```

The program evaluates the part (b) expression and the part (d) minimized expression for all sixteen input combinations, simulates the part (e) NAND circuit, and compares all three against the original gate network.

Traced by hand for one input row, $A = 1$, $B = 0$, $C = 1$, $D = 1$, so $C' = 0$ and $D' = 0$:

Token	Action	Values (bottom to top)	Operators
`B`	push 0	0	
`&`	push `&`	0	`&`
`D'`	push 0	0 0	`&`
`|`	apply $0 \wedge 0 = 0$, push `|`	0	`|`
`B`	push 0	0 0	`|`
`&`	push `&`	0 0	`| &`
`C`	push 1	0 0 1	`| &`
`|`	apply $0 \wedge 1 = 0$, then $0 \vee 0 = 0$, push `|`	0	`|`
`A`	push 1	0 1	`|`
`&`	push `&`	0 1	`| &`
`D`	push 1	0 1 1	`| &`
`|`	apply $1 \wedge 1 = 1$, then $0 \vee 1 = 1$, push `|`	1	`|`
`A`	push 1	1 1	`|`
`&`	push `&`	1 1	`| &`
`C'`	push 0	1 1 0	`| &`

Remaining calculations: $1 \wedge 0 = 0$ leaving values 1 0, then $1 \vee 0 = 1$.

$\mathbf{Z = 1}$, which matches the row $ABCD = 1011$ of the truth table in part (c).

Every time a `|` arrives, the waiting `&` is applied first, just as `*` runs before `+` in Problem 1.

Code

```
// Q2: evaluating the circuit expression using two stacks

#include <iostream>
#include <iomanip>
#include <string>
#include <vector>
using namespace std;

const int MAX = 100;

class ValueStack
{
    int a[MAX];
    int t = -1;
public:
```

```
    bool empty() { return t == -1; }
    bool full() { return t == MAX - 1; }
    void push(int x) { if (!full()) a[++t] = x; }
    int pop() { return a[t--]; }
};

class OpStack
{
    char a[MAX];
    int t = -1;
public:
    bool empty() { return t == -1; }
    bool full() { return t == MAX - 1; }
    char top() { return a[t]; }
    void push(char c) { if (!full()) a[++t] = c; }
    char pop() { return a[t--]; }
};

bool isVariable(string s)
{
    return s[0] >= 'A' && s[0] <= 'D';
}

int valueOf(string s, int A, int B, int C, int D)
{
    int v = 0;
    if (s[0] == 'A') v = A;
    else if (s[0] == 'B') v = B;
    else if (s[0] == 'C') v = C;
    else v = D;
    if (s.size() > 1 && s[1] == '\'') v = 1 - v;
    return v;
}

int prec(char c)
{
    if (c == '&') return 2;
    return 1;
}

int applyOp(int a, int b, char c)
{
    if (c == '&') return a & b;
    return a | b;
}

int nand2(int a, int b) { return 1 - (a & b); }

int evaluate(vector<string> tok, int A, int B, int C, int D)
{
    ValueStack vals;
    OpStack ops;

    for (int i = 0; i < (int)tok.size(); i++)
    {
        if (isVariable(tok[i]))
        {
            vals.push(valueOf(tok[i], A, B, C, D));
        }
```

```
        else
        {
            char c = tok[i][0];
            while (!ops.empty() && prec(ops.top()) >= prec(c))
            {
                char o = ops.pop();
                int b = vals.pop();
                int a = vals.pop();
                vals.push(applyOp(a, b, o));
            }
            ops.push(c);
        }
    }

    while (!ops.empty())
    {
        char o = ops.pop();
        int b = vals.pop();
        int a = vals.pop();
        vals.push(applyOp(a, b, o));
    }

    return vals.pop();
}

int main()
{
    vector<string> Zb   = {"A", "&", "B", "|", "A", "&", "C'", "|", "B", "&", "C",
                           "|", "B", "&", "D'", "|", "A", "&", "D"};
    vector<string> Zmin = {"B", "&", "D'", "|", "B", "&", "C",
                           "|", "A", "&", "D", "|", "A", "&", "C'"};

    cout << " A B C D | N1 N2  P  Q  R  S  T |  Z gates  Z (b)  Z (d)  Z nand" << endl
        ;
    cout << "---------+----------------------+------------------------------" << endl;

    bool allMatch = true;
    for (int A = 0; A <= 1; A++)
      for (int B = 0; B <= 1; B++)
        for (int C = 0; C <= 1; C++)
          for (int D = 0; D <= 1; D++)
          {
              int N1 = !C, N2 = !D;
              int P = B | N1, Q = A & P;
              int R = C | N2, S = B & R;
              int T = A & D;
              int Zgates = Q | S | T;

              int Zexpanded = evaluate(Zb, A, B, C, D);
              int Zminimal  = evaluate(Zmin, A, B, C, D);

              int Cn = nand2(C, C), Dn = nand2(D, D);
              int g1 = nand2(B, Dn), g2 = nand2(B, C);
              int g3 = nand2(A, D),  g4 = nand2(A, Cn);
              int X = nand2(g1, g2), Y = nand2(g3, g4);
              int Znand = nand2(nand2(X, X), nand2(Y, Y));

              if (Zgates != Zexpanded || Zgates != Zminimal || Zgates != Znand)
                  allMatch = false;
```

```
                cout << " " << A << " " << B << " " << C << " " << D << " |"
                     << setw(3) << N1 << setw(3) << N2 << setw(3) << P
                     << setw(3) << Q  << setw(3) << R  << setw(3) << S
                     << setw(3) << T  << " |" << setw(6) << Zgates
                     << setw(8) << Zexpanded << setw(7) << Zminimal
                     << setw(8) << Znand << endl;
            }

    cout << endl;
    cout << "Z (b) = A & B | A & C' | B & C | B & D' | A & D" << endl;
    cout << "Z (d) = B & D' | B & C | A & D | A & C'" << endl << endl;
    cout << "expressions and NAND circuit match the gate circuit on all 16 rows: ";
    cout << (allMatch ? "yes" : "no") << endl;
    return 0;
}
```

Output

```
 A B C D | N1 N2  P  Q  R  S  T |  Z gates  Z (b)  Z (d)  Z nand
---------+----------------------+-----------------------------
 0 0 0 0 |  1  1  1  0  1  0  0 |     0       0       0       0
 0 0 0 1 |  1  0  1  0  0  0  0 |     0       0       0       0
 0 0 1 0 |  0  1  0  0  1  0  0 |     0       0       0       0
 0 0 1 1 |  0  0  0  0  1  0  0 |     0       0       0       0
 0 1 0 0 |  1  1  1  0  1  1  0 |     1       1       1       1
 0 1 0 1 |  1  0  1  0  0  0  0 |     0       0       0       0
 0 1 1 0 |  0  1  1  0  1  1  0 |     1       1       1       1
 0 1 1 1 |  0  0  1  0  1  1  0 |     1       1       1       1
 1 0 0 0 |  1  1  1  1  1  0  0 |     1       1       1       1
 1 0 0 1 |  1  0  1  1  0  0  1 |     1       1       1       1
 1 0 1 0 |  0  1  0  0  1  0  0 |     0       0       0       0
 1 0 1 1 |  0  0  0  0  1  0  1 |     1       1       1       1
 1 1 0 0 |  1  1  1  1  1  1  0 |     1       1       1       1
 1 1 0 1 |  1  0  1  1  0  0  1 |     1       1       1       1
 1 1 1 0 |  0  1  1  1  1  1  0 |     1       1       1       1
 1 1 1 1 |  0  0  1  1  1  1  1 |     1       1       1       1

Z (b) = A & B | A & C' | B & C | B & D' | A & D
Z (d) = B & D' | B & C | A & D | A & C'

expressions and NAND circuit match the gate circuit on all 16 rows: yes
```